Neutral Network Report

Analysis and Report

Method.

The baseline was a 784,128,64,10 MLP trained with plain SGD (lr 0.01, no dropout) for 10 epochs. For the experiment I varied three hyperparameters in a full grid of 12 runs: learning rate (0.001, 0.01, 0.1), optimiser (SGD, Adam) and dropout (0.0, 0.3). The first two belong to the learning process and dropout is regularisation. Every run used the same seed, data split and 10 epochs, so differences come from the hyperparameters. Results are the final-epoch validation accuracy.

Results summary

| Setting | Average validation accuracy per setting |
|---|---|
| lr 0.001 / 0.01 / 0.1 | 0.831 / 0.853 / 0.476 |
| SGD / Adam | 0.829 / 0.611 |
| Dropout 0.0 / 0.3 | 0.722 / 0.718 |

Best vs baseline.

The best model used Adam, learning rate 0.001 and no dropout, reaching 0.8870 validation accuracy against 0.8586 for the baseline. That is an improvement of 2.84 percentage points, cutting the error rate from 14.1% to 11.3%. Three other runs also beat the baseline, Adam 0.001 with dropout 0.3 at 0.8828, Adam 0.01 at 0.8665 and SGD 0.1 with dropout at 0.8698. Adam adapts the step size of each weight, so it makes much more progress than plain SGD in 10 epochs.

Hyperparameter influence.

Learning rate had the biggest effect. At lr 0.1, both Adam runs collapsed to about 10% accuracy, which is random guessing for 10 classes, with the loss still at about 2.30. The steps were too large, so the weights never settled. This pulls the lr 0.1 average down to 0.476, against 0.853 at lr 0.01. The best learning rate depends on the optimiser. Adam works best with a small rate (0.001), because it already scales its own steps. Plain SGD needs a larger rate, at 0.001 it was still improving at epoch 10 (0.7887), so it was too slow, while at 0.1 it reached 0.8698 with dropout. Dropout had little effect on average (0.722 vs 0.718). The baseline barely overfits, so there was little for dropout to fix, and it

slows learning slightly in 10 epochs. It helped most for SGD at lr 0.1, where it improved accuracy from 0.8334 to 0.8698 and steadied the validation loss.

Overfitting analysis.

| | Final train loss | Final val loss | Gap |
|---|---|---|---|
| Baseline | 0.352 | 0.390 | 0.038 |
| Best model | 0.217 | 0.327 | 0.110 |

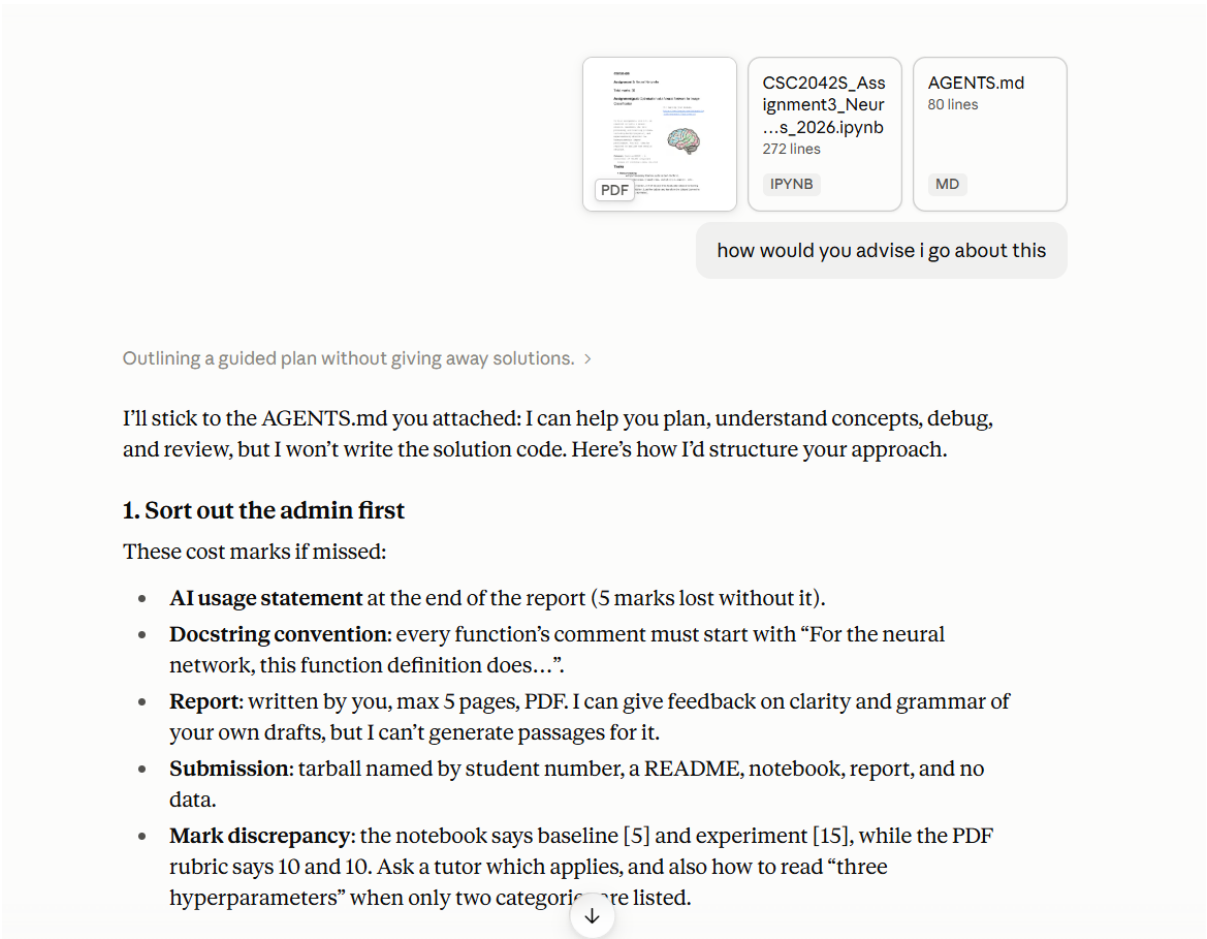
On the loss curves, the baseline's dashed (train) and solid (validation) lines stay close together and are both still falling, so it shows very little overfitting and has probably not finished learning. The best model's validation loss is lower, but the gap between its curves is wider. Its training loss keeps dropping while its validation loss flattens and wiggles after about epoch 6, which is mild overfitting. So tuning mainly improved accuracy and speed of learning. It did not reduce overfitting. Adding dropout 0.3 to the same setup did remove the gap (train 0.325, val 0.325), at a small cost in accuracy (0.8828 vs 0.8870).

Error analysis.

The most frequent confusions in the confusion matrix are Shirt predicted as T-shirt/top (173) and T-shirt/top predicted as Shirt (81). Both are upper-body garments with nearly the same torso outline, and at 28×28 greyscale the features that separate them (collar, buttons, sleeve length) cover only a few pixels. Many shirts also have short sleeves and plain fabric, so they overlap with T-shirts. A second common confusion is Pullover predicted as Coat (91) (and as Shirt, 97),all are long-sleeved with similar shapes, and a coat's zip or buttons are hard to see at this size. An MLP flattens the image, so it has no notion of which pixels are neighbours and struggles to pick up small local details like collars.

Conclusion. Choosing a learning rate that suits the optimiser mattered far more than dropout. The best configuration (Adam, lr 0.001) improved on the baseline by 2.84, and its remaining errors come mostly from visually similar clothing classes.

AI Usage:




I used AI to help me come up with a roadmap on how to think around the assignment and I asked for help on how to debug the graphs and visualisations.